🧠 OD PROMPT ENGINEERING DO CONTEXT ENGINEERING

- 2023: „Jak napisać lepszy prompt?"
- 2024: „Jak zbudować system promptów z rolami i szablonami?"
- 2026: „Jak dostarczyć modelowi **właściwy kontekst** – narzędzia, dane, pamięć?"

🎯 Context Engineering = sztuka budowania **całego środowiska**, w którym model pracuje

🧩 CO SKŁADA SIĘ NA KONTEKST MODELU?

📋 SYSTEM PROMPT
Rola, zasady, ograniczenia

🔧 TOOLS (NARZĘDZIA)
API, bazy danych, wyszukiwarki

📚 RAG / WIEDZA
Dokumenty, procedury, baza wiedzy

💾 PAMIĘĆ
Historia rozmowy, stan zadania

🔧 TOOL CALLING – MODEL UŻYWA NARZĘDZI

Model nie „wie" wszystkiego – ale może **wywołać funkcję**, żeby się dowiedzieć.

```
tools = [{
  "type": "function",
  "function": {
    "name": "classify_ticket",
    "description": "Classify an IT ticket",
    "parameters": {
      "properties": {
        "category": {"enum": ["Hardware","Network","Software","Account"]},
        "confidence": {"type": "number"},
        "reasoning": {"type": "string"}
}}}}]
```

🤖 Model widzi narzędzia → decyduje, które wywołać → zwraca structured output

📐 STRUCTURED OUTPUTS – JSON MODE

❌ 2023

```
Prompt: "Classify this tick

Output: "I would classify t
as Network because..."
```

→ Parsowanie tekstu, regex...

✅ 2026

```
{
  "category": "Network",
  "confidence": 0.85,
  "reasoning": "Printer via
}
```

→ Gotowy JSON, zero parsowania

🌡️ PARAMETRY MODELU – CO KONTROLUJEMY?

- **Temperature** – 0 = deterministic, 1.5 = kreatywny chaos
  Klasyfikacja = 0, brainstorm = 1.0+
- **Max tokens** – limit długości odpowiedzi
  Krótka odpowiedź = 100, esej = 4000
- **Context window** – ile tekstu model „pamięta"
  GPT-4o: 128K, Claude: 200K, Gemini: 2M tokenów
- **System prompt** – „osobowość" i reguły modelu
  Kim jest, jak się zachowuje, czego nie wolno

🧱 TYPY MODELI JĘZYKOWYCH

- **Foundation model:** ogólny, ogromny, bez specjalizacji (np. GPT-4o, Claude, Gemini)
- **Fine-tuned:** ten sam model, ale podciągnięty pod konkretny temat (np. model do analizy CV)
- **RAG:** najpierw szuka informacji w dokumentach, potem odpowiada

📊 KIEDY UŻYĆ KTÓREGO?

- **Foundation:** kreatywne zadania, testowanie, „pogadaj z AI"
- **Fine-tuned:** dokładność, konkretne zadania, np. CV, analiza logów
- **RAG:** chatbot firmowy, AI z dostępem do dokumentacji

QUIZ: TYPY MODELI LLM

Reaguj na Zoomie:

👍 = Foundation | 🎉 = Fine-tuned | 🤫 = RAG

QUIZ 1/6

📋 Agent AI w firmie sprawdza Confluence i Jira, zanim odpowie na pytanie pracownika

👍 Foundation | 🎉 Fine-tuned | 🤫 RAG

QUIZ 1/6 — ODPOWIEDŹ

📋 Agent AI z dostępem do Confluence i Jira

➡️ 🤫 **RAG**

QUIZ 2/6

🤖 Cursor / Claude Code generuje kod na podstawie opisu w komentarzu

👍 Foundation | 🎉 Fine-tuned | 🤫 RAG

QUIZ 2/6 — ODPOWIEDŹ

🤖 Cursor / Claude Code — generowanie kodu

➡️👍 **Foundation**

QUIZ 3/6

🎯 System klasyfikujący zgłoszenia IT na podstawie 50 000 historycznych ticketów

👍 Foundation | 🎉 Fine-tuned | 🤫 RAG

QUIZ 3/6 — ODPOWIEDŹ

🎯 Klasyfikator IT — 50K ticketów treningowych

➡️🎉 **Fine-tuned**

QUIZ 4/6

📝 Claude generuje podsumowanie spotkania z wklejonej transkrypcji

👍 Foundation | 🎉 Fine-tuned | 🤫 RAG

QUIZ 4/6 — ODPOWIEDŹ

📝 Claude — podsumowanie z transkrypcji

➡️👍 **Foundation**

QUIZ 5/6

🏥 Model wytrenowany na 100K raportów klinicznych do klasyfikacji efektów ubocznych leków

👍 Foundation | 🎉 Fine-tuned | 🤫 RAG

QUIZ 5/6 — ODPOWIEDŹ

🏥 Klasyfikator efektów ubocznych — pharma

➡️🎉 **Fine-tuned**

QUIZ 6/6

🛒 Chatbot e-commerce przeszukuje katalog 10K produktów i historię zamówień klienta

👍 Foundation | 🎉 Fine-tuned | 🤫 RAG

QUIZ 6/6 — ODPOWIEDŹ

🛒 Chatbot e-commerce — katalog + zamówienia

➡️ 🤫 **RAG**

🧠 ZASADA NA WYNOS

- **Dane w prompcie** → Foundation (lub RAG, jeśli są pobierane automatycznie)
- **Dane zmieniają wagi modelu** → Fine-tuned
- **Model szuka danych przed odpowiedzią** → RAG

💡 Większość firm w 2026 zaczyna od RAG — nie wymaga trenowania, łatwo aktualizować dane

📦 POPULARNE MODELE LLM – 2026

Model	Firma	Charakterystyka	Lokalnie?
GPT-4o	OpenAI	Multimodalny, szybki, popularny	❌ API
Claude 4	Anthropic	200K kontekst, bezpieczny, analityczny	❌ API
Gemini 2.5	Google	2M kontekst, integracja z Google	❌ API
Gemma 3	Google	4B–27B, multimodalny, świetny lokalnie	✅ Ollama
Llama 4	Meta	Open source, duży ekosystem	✅ Ollama
Phi-4	Microsoft	Mały, szybki, świetny do zadań specjalistycznych	✅ Ollama

⚔️ ĆWICZENIE: CHATBOT ARENA

Porównajmy modele na żywo – blind test!

👉 lmarena.ai

- Wpiszcie prompt → dostajecie 2 odpowiedzi od nieznanych modeli
- Wybierzcie lepszą → na końcu zobaczcie, jakie to były modele
- 🎯 Pytanie: „Classify this IT ticket: Network printer not responding"

⏱️ 5 minut – potem porównujemy wyniki na czacie!

🎨 VIBE CODING — BUDUJEMY RAZEM!

AI pisze kod. **Wy decydujecie co.**

Format: 4 rundy decyzji

Każda runda → głosujecie reakcjami Zoom → trener promptuje AI

Na koniec → działająca aplikacja zbudowana w ~20 minut

🛠️ Narzędzie: Claude Code / Cursor (trener na ekranie)

🎨 RUNDA 1: CO BUDUJEMY?

Głosujcie reakcjami Zoom!

👍 **Dashboard zgłoszeń IT** — tabela, filtry, statusy

🎉 **Wyszukiwarka bazy wiedzy** — search bar + wyniki

😲 **Monitor statusu systemów** — karty, kolory, uptime

🎨 RUNDA 2: JAKI FRAMEWORK?

Głosujcie!

👍 **Streamlit** — Python, prosty, idealny do prototypów

🎉 **Flask + HTML** — klasyka, pełna kontrola

🤫 **Czysty HTML/JS** — zero backendu, od razu w przeglądarce

🎨 RUNDA 3: PIERWSZA FUNKCJA?

Głosujcie!

👍 **Wykresy i statystyki** — ile zgłoszeń per kategoria

🎉 **Wyszukiwanie** — filtruj po tekście, kategorii

😲 **Formularz dodawania** — nowe zgłoszenie z poziomu UI

🎨 RUNDA 4: FINISHING TOUCH?

Ostatnie głosowanie!

👍 **Dark mode** — ciemny motyw

🎉 **Eksport CSV** — pobierz dane

🤫 **AI klasyfikator** — wpisz zgłoszenie, AI kategoryzuje

🎯 Aplikacja gotowa! Od zera do działającego narzędzia w ~20 minut.

🛠️ ĆWICZENIE: BUDUJEMY SPECJALISTĘ Z OLLAMA MODELFILE

Stwórzmy własnego bota IT – z nazwą, osobowością i parametrami!

```
FROM gemma3:4b
PARAMETER temperature 0
SYSTEM """You are ITBot, an IT helpdesk assistant.
- Always respond in Polish
- Classify into: Sprzet, Siec, Oprogramowanie, Konto
- Explain reasoning, suggest next step"""
```

```
ollama create itbot -f /tmp/ITBot && ollama run itbot
```

🎯 Przetestuj: „Drukarka sieciowa nie odpowiada po awarii prądu"

🤔 CO LLM UMIE – A CZEGO NIE?

- ✅ Generowanie tekstu, kodu, podsumowań, tłumaczenie
- ✅ Klasyfikacja, analiza sentymentu, ekstrakcja danych
- ❌ Nie „rozumie" świata – halucynacje, wymyślanie faktów
- ❌ Nie ma pamięci między sesjami (chyba że mu ją zbudujesz!)
- ⚠️ Context Engineering rozwiązuje wiele tych ograniczeń

🤖 OD MODELI DO AGENTÓW AI

LLM to potężny silnik… ale sam nie wie, co ma robić 🚗

- **Agent = LLM + Tools + Memory + Planning**
- Model decyduje, które narzędzie użyć
- Pamięta kontekst rozmowy
- Planuje kolejne kroki

🎯 To jest kierunek, w którym idzie cała branża AI w 2026

📚 KOMPONENTY AGENTA AI

- 🧠 **Model językowy (LLM):** rozumienie instrukcji, podejmowanie decyzji
- 🔧 **Narzędzia (Tools):** API, bazy danych, funkcje systemowe
- 💾 **Pamięć:** krótkoterminowa (rozmowa) i długoterminowa (baza wiedzy)
- 📋 **Planowanie:** podział zadania na kroki, ustalanie kolejności
- 👁️ **Monitorowanie:** kontrola poprawności i bezpieczeństwa

💬 VS 🤖 — CHATBOT VS AGENT

💬 CHATBOT

Uzytkownik pyta

↓

LLM generuje tekst

↓

Odpowiedz tekstowa

"Twoje zgloszenie dotyczy sieci."

🤖 AGENT

Uzytkownik pyta

↓

LLM **decyduje** i uzywa narzedzi

↓

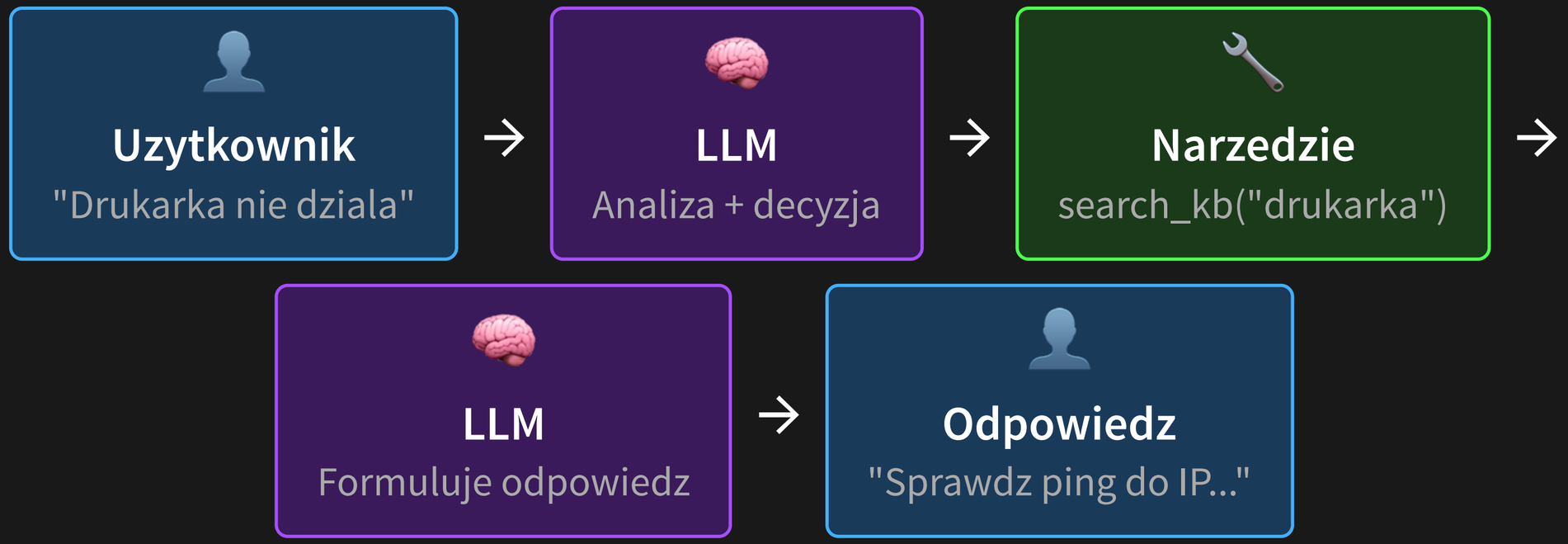
Sprawdza baze wiedzy, tworzy ticket

↓

Odpowiedz + **wykonana akcja**

"Sprawdzilem ping — brak. Ticket T

🔄 PETLA AGENTA — JAK TO DZIALA?



Kluczowe: LLM jest wywolywany **2 razy** — raz na decyzje ("uzyc narzedzia?"), raz na odpowiedz

🌍 PRZYKŁADY AGENTÓW AI W PRAKTYCE

- 📩 **IT Helpdesk:** klasyfikacja, baza wiedzy, eskalacja
- 📈 **Asystent sprzedażowy:** analiza leadów, oferty, follow-upy
- 🔍 **Code Review:** pull requesty, poprawki, bezpieczeństwo
- 📊 **Data Analyst:** SQL, raporty, wizualizacje
- 🛡️ **Security:** monitoring logów, anomalie, eskalacja

🌟 W 2026 agenci to codzienne narzędzia: Claude Code, Copilot, Cursor

🔀 WZORCE ORKIESTRACJI AGENTOW

Nie kazdy agent to ta sama architektura. Oto 4 glowne wzorce:

🚦 ROUTER

Klasyfikuj → wybierz sciezke → wykonaj

Analogia: **recepcja hotelowa** — goscie kierowani do wlasciwego pietra

Helpdesk, triage, routing

📋 PLAN-AND-EXECUTE

Jeden model planuje, tani model wykonuje

Analogia: **architekt + ekipa budowlana** — plan rysuje drogi, buduje tani

Redukcja kosztow o 90%

🪞 REFLECT

Agent sprawdza swoj wynik i poprawia

Analogia: **redaktor prasowy** — pisze, czyta, poprawia, czyta znowu

QA, code review, recenzje

👤 HUMAN-IN-THE-LOOP

Agent pyta czlowieka w kluczowych momentach

Analogia: **pilot automatyczny** — leci sam, ale laduje pilot

Finanse, medycyna, prawo

🍳 ORKIESTRACJA W KUCHNI RESTAURACJI

Agenci AI dzialaja jak dobrze zorganizowana kuchnia:

📋

Kelner

Zbiera zamowienie

(Input)

→

👨‍🍳

Szef kuchni

Decyduje co robic

(LLM)

→

🍳🥗🔥

Kucharze

Specjalisci od dan

(Narzedzia)

→

✅

Wydanie

Kontrola jakosci

(Monitoring)

Kluczowe: Szef kuchni nie gotuje — **koordynuje**. LLM nie wykonuje akcji — **decyduje**, ktore narzedzie wykonuje.

📝 ĆWICZENIE: ZAPROJEKTUJ SWOJEGO AGENTA

W grupach 2-3 osoby – 10 minut:

1. Wybierz problem z pracy, który agent mógłby rozwiązać
2. Narysuj: **Input → Analiza → Decyzja → Akcja**
3. Określ: narzędzia? Pamięć? Reguły?
4. Przedstaw w 1 minucie na czacie

Przykład: Agent HR — CV (input) → analiza doświadczenia (LLM) → wymagania (tool: baza HR) → zaproszenie/odrzucenie → mail (action)

🍽️ PRZERWA OBIADOWA – 60 MINUT

Wracamy o 13:45

Po przerwie: budujemy prawdziwego agenta w Pythonie! 🐍

PRZEJDŹ DO CZĘŚCI 3

➡️ Część 3: Warsztaty praktyczne i Podsumowanie